

Activitate 1 - Expresii regulate

1. Sumar

Etapele scrierii unui compilator

Analiza lexicală

- Preia caracterele individuale din codul sursă.
- Le grupează în **atomi lexicali**: numere, identificatori, operatori etc.

Atom lexical = componenta primară, indivizibilă, utilizată ulterior de compilator.

În această etapă sunt eliminate elementele care nu sunt necesare pentru compilare:

- spații
 - comentarii
 - separatori redundanți
-

Analiza sintactică

Asamblează atomii lexicali în **structuri gramaticale ale limbajului**.

Exemplu:

O expresie este formată din:

- operanzi
 - operatori
 - paranteze
- acestea fiind aranjate conform regulilor limbajului.
-

Analiza de domeniu (scope analysis)

Asociază fiecare **identificator** cu definiția sa.

Exemplu în limbajul C:

- o variabilă locală este accesibilă **doar în interiorul funcției** în care este definită.
-

Analiza de tipuri

Determină tipurile de date utilizate în expresii.

Exemplu în C:

``int + float → float`

Adunarea dintre un număr întreg și unul zecimal produce un rezultat **zecimal**.

Mașina virtuală

Reprezintă limbajul intermediar generat de compilator.

Există două posibilități:

1. cod specific unui **microprocesor real** (cod mașină)
2. cod pentru o **mașină virtuală**

Exemple de limbaje care folosesc mașini virtuale:

- Java
 - C#
-

Generarea de cod

Etapă finală a compilării.

Codul sursă este translatat în **instrucțiuni executabile**.

În această etapă pot fi aplicate **optimizări**.

Exemplu:

``1 + 2`

este optimizat direct în

``3`

Această optimizare se numește **constant folding**.

Expresii regulate (Regex)

O **expresie regulată** este un tipar folosit pentru a identifica secvențe de caractere într-un text.

Caractere de bază

`a` → caracter literal

. → orice caracter

Clase de caractere

[abc] → a sau b sau c
[a-z] → litere mici
[A-Z] → litere mari
[0-9] → cifre
[a-zA-Z] → orice literă
[^abc] → orice caracter în afară de a,b,c

Operatori

e1e2 → secvență (ambele trebuie să apară)
e1|e2 → alternativă (sau)
(e) → grupare

Repetiții

e? → 0 sau 1 apariții (opțional)
e* → 0 sau mai multe apariții
e+ → 1 sau mai multe apariții
e{n} → exact n apariții
e{n,m} → între n și m apariții

Ancore

^ → începutul șirului
\$ → sfârșitul șirului

Escape

\.
*
\?
\\

\ elimină semnificația specială a caracterului.

Exemplu:

`3.14

Exemple utile

Identificator:

```
[a-zA-Z_][a-zA-Z0-9_]*
```

Număr zecimal:

```
[0-9]+(\.[0-9]+)?
```

Cuvinte:

```
[a-zA-Z]+
```

✓ **Ordinea de prioritate a operatorilor:**

1 * + ? {}

2 concatenare

3 |

2. Exerciții:

Regex site: [regex101: build, test, and debug regex](https://regex101.com)

Aplicația 1.1: Într-un text există date calendaristice de forma „zi/luna/an”, în care ziua și luna sunt formate din 1-2 cifre iar anul din 2 sau 4 cifre. Să se scrie o expresie regulată care găsește toate aceste date. Nu este necesar să se facă validarea datelor calendaristice.

Ex: Ion s-a născut pe data de 21/5/1990 și Maria pe data de 7/12/92.

Rezolvare:

Din enunț, data poate fi determinată ca fiind de forma:

```
zz/ll/aaaa sau zz/ll/aa
```

Și avem următoarele restricții:

- 01|1 <= zz <= 31
- 01|1 <= ll <= 12

Pentru zi, urmărim logica:

```
01|1 -> 31
```

```
01 - 09 => 0?[1-9] - adică 0 poate apărea de 0 sau 1 ori, urmat de o cifră
```

```
10 - 29 => [12][0-9]
```

```
30 - 31 => [3][01]
```

Logica pentru delimitatoare este următoarea: avem . sau \ și expresia este `(.|/).

Regex-ul final pentru **zi** este:

```
0?[1-9]| [12][0-9]| [3][01]
```

În aceeași manieră, deducem pentru lună:

```
01 - 09 => 0?[1-9]
10 - 12 => [1][0-2]
```

Regex-ul final pentru **lună** este:

```
0?\[1-9]|[1][0-2].
```

Pentru **an** avem:

```
aaaa | aa:
```

`[0-9]{4}|[0-9]{2}` - aici am pus opțiunea de potrivire fie pentru 4 cifre, fie pentru două, în această ordine, deoarece regex-ul este greedy și dacă ar fi invers, atunci ar face un match greșit.

Forma finală este:

```
(0?[1-9]| [12][0-9]| 3[01])(\.|\/)(0?[1-9]| [1][0-2])(\.|\/)([0-9]{4}|[0-9]{2})
```

Aplicația 1.2: Propuneți un text în care există o secvență de caractere care corespunde expresiei regulate:

`[A-Z][a-zA-Z],[a-z]+ [0-9]+(/.)(.|?)`

Rezolvare:

```
[A-Z] - literă mare, 1 apariție
[a-zA-Z]* - literă mică sau literă mare, 0 sau mai multe apariții
, - virgulă, o apariție
[a-z]+ - literă mică, 1 sau mai multe apariții
    - spațiu, o apariție
[0-9]+ - cifră, 1 sau mai multe apariții
(\./)* - 0 sau mai multe secvențe de tip / urmat de orice caracter
(\.|\/?) - punct sau semnul întrebării, o apariție
```

Exemple:

```
A,a 1?  
Maria,ion 42/.?  
Ana,are 12/2.
```

Aplicația 1.3: Să se scrie o expresie regulată care testează o sintaxă simplificată pentru clasele de caractere. Acestea trebuie să înceapă cu „[” și să se termine cu „]”; opțional după „[” poate fi „^”; în interior pot fi caractere individuale sau intervale de caractere; un caracter poate fi simplu sau prefixat de „\”.

Exemple valide: [abc] [^0-9] [\r\n\t] [., !?;-] [\]

Exemple invalide (nu trebuie recunoscute): [] [\] [a

Rezolvare:

```
pentru „[” -> \[  
pentru „]” -> \]  
pentru „^” -> \^? - 0 sau 1 apariție(optional)  
pentru interiorul clasei -> caracterul „\” urmat de orice caracter: \\  
-> orice caracter, mai puțin „]” și „\” și „-”:  
[^\]\\-].  
-> interval de caractere: [^\]\\-[^\\]\\]
```

Expresia finală:

```
\\[\\^?(\\.|([^\]\\-])|([^\]\\-[^\\]\\-)))+\\]
```